Universidade Estadual do Piauí (UESPI), Campus Piripiri
Curso de Computação, Segurança Computacional, 2026

Auditoria de TV Boxes Android apreendidas

Relatório técnico do Grupo 2: análise dinâmica e de rede

Unidade analisada: TV box Allwinner H3, placa `dolphin-fvd-p1`, serial `14f2782704d344000052`
Data dos experimentos: 9 de outubro de 2026, das 10:10 às 13:20 (horário de Brasília)
Integrantes: João Marcello e Vinícius Dias

Resumo

Este relatório descreve a análise dinâmica de uma TV box Android apreendida. A unidade foi colocada em uma rede Wi-Fi isolada, sem acesso à internet e sem acesso à rede local. Foram observados processos, portas abertas, consultas DNS e tentativas de conexão em quatro situações: ociosa, logo após a reinicialização, durante o uso por uma pessoa e com um servidor falso (sinkhole) no lugar de um domínio de atualização.

A unidade declara Android 11.1 e 8 GB de RAM. Os dados internos mostram Android 7.0 (API 24) com patch de segurança de novembro de 2016 em um SoC Allwinner H3 de 32 bits. O firmware é uma compilação de desenvolvimento (`eng`, `test-keys`) com SELinux em modo permissivo e ADB com acesso root sem autenticação.

O principal achado dinâmico envolve dois aplicativos de sistema, `com.uv.droid.uota` e `com.uv.droid.uotaback`, que executam com UID 1000 (system). Eles consultam o domínio `uota.uota.xyz` e tentam conexão HTTP na porta 2082. Com o sinkhole foi possível registrar a requisição completa: os dois apps enviam em texto claro o MAC do Wi-Fi, o número de série do processador, o Android ID e a identificação do firmware. Isso foi classificado como **vulnerável/mal configurado** (identificadores persistentes e canal de atualização privilegiado sem TLS). Não foi demonstrado comportamento malicioso, porque a resposta do servidor real não foi observada.

Também foram observados contatos da loja alternativa "GetApps" (pacote `com.simple.appstore.flymarket`) com `appstore.bb20.site` na porta 2086 e com o SDK Tencent Bugly. Os apps UOTA também tentam, em intervalos de cerca de 7 min 30 s, conexões HTTP com `www.baidu.com` e `www.google.com`. A associação ao Baidu foi feita no experimento E5, com leitura das conexões três vezes por segundo. A análise estática do Grupo 1 mostrou que essas conexões servem de fonte de hora (cabeçalho HTTP `Date`). No experimento E6 foi feita uma auditoria cruzando todos os destinos capturados com os processos observados. Ela revelou que o par UOTA também envia pedidos de hora (NTP) para uma lista própria de servidores, que inclui seis endereços IP fixos no código. Ao final, todos os destinos externos observados têm processo associado, por observação direta ou pelo domínio. O aplicativo `com.qcode.ad_service_server` não executou em nenhum dos experimentos.

O relatório do Grupo 1 confirmou no código o que este relatório só podia inferir: a resposta do servidor de atualização pode levar a download e instalação silenciosa de pacotes, validada apenas por um MD5 recebido pelo mesmo canal HTTP. O Grupo 3 desativou os atualizadores e o serviço de controle remoto em outra unidade com o mesmo firmware: os processos do UOTA e as portas expostas desapareceram, mas o tráfego após o hardening não foi capturado. A seção 9 cruza os resultados dos três grupos.

Sumário

1: Objetivo e escopo 4

2: Ambiente de teste 5

2.1: Regras de isolamento 5

2.2: Acesso ao sistema 6

3: Caracterização da unidade 7

4: Metodologia 8

4.1: Ferramentas 8

4.2: Correlação conexão-processo 8

4.3: Experimentos 9

4.4: Critério de classificação 10

5: Resultados por experimento 11

5.1: E1 e E1b: box ociosa 11

5.2: E2: reinicialização 11

5.3: E3: sinkhole de uota.uota.xyz 12

5.4: E4: uso por uma pessoa 14

5.5: E5: origem da consulta ao Baidu 14

5.6: E6: auditoria de cobertura e NTP com IPs fixos 15

6: Tabela consolidada 17

7: Comportamentos explicados e não explicados 20

8: Matriz de achados 21

9: Correlação com os Grupos 1 e 3 23

9.1: Medidas do Grupo 3 e efeito observado 24

9.2: Um caso visto pelos três grupos 25

9.3: Pendências da correlação 25

9.4: Recomendações conjuntas 25

10: Limitações e desvios de procedimento 26

11: Alterações realizadas na unidade 28

12: Conclusão 29

Apêndice A: evidências e hashes 30

Apêndice B: requisição capturada no sinkhole 31

Apêndice C: scripts utilizados 32

1: Objetivo e escopo

Conforme a seção 9 da proposta de atividade, o objetivo do Grupo 2 é observar o comportamento real do equipamento em execução, com atenção a processos, serviços, portas, DNS e comunicações externas. As atividades realizadas cobrem:

- registro de processos e serviços após o boot e após períodos de ociosidade;
- identificação de portas TCP/UDP abertas;
- captura de consultas DNS e tentativas de conexão externas;
- comparação entre execução ociosa e execução com interação do usuário;
- observação após reinicialização, com repetição dos experimentos;
- correlação de conexões a processos e pacotes Android.

Ficaram fora do escopo a análise do conteúdo dos APKs (Grupo 1) e qualquer alteração permanente na unidade (Grupo 3). A unidade será usada depois pelo Grupo 1, por isso foi adotada a regra de não fazer alterações persistentes. A seção 11 lista tudo o que foi feito na unidade.

2: Ambiente de teste

Não havia cabo Ethernet disponível. Por isso o notebook de análise foi usado como ponto de acesso Wi-Fi. Ele manteve a própria conexão com a rede doméstica e criou uma segunda interface virtual (`ap0`) em modo AP, no mesmo canal (149, 5 GHz), já que a placa Intel AX200 só permite um canal ativo por vez.

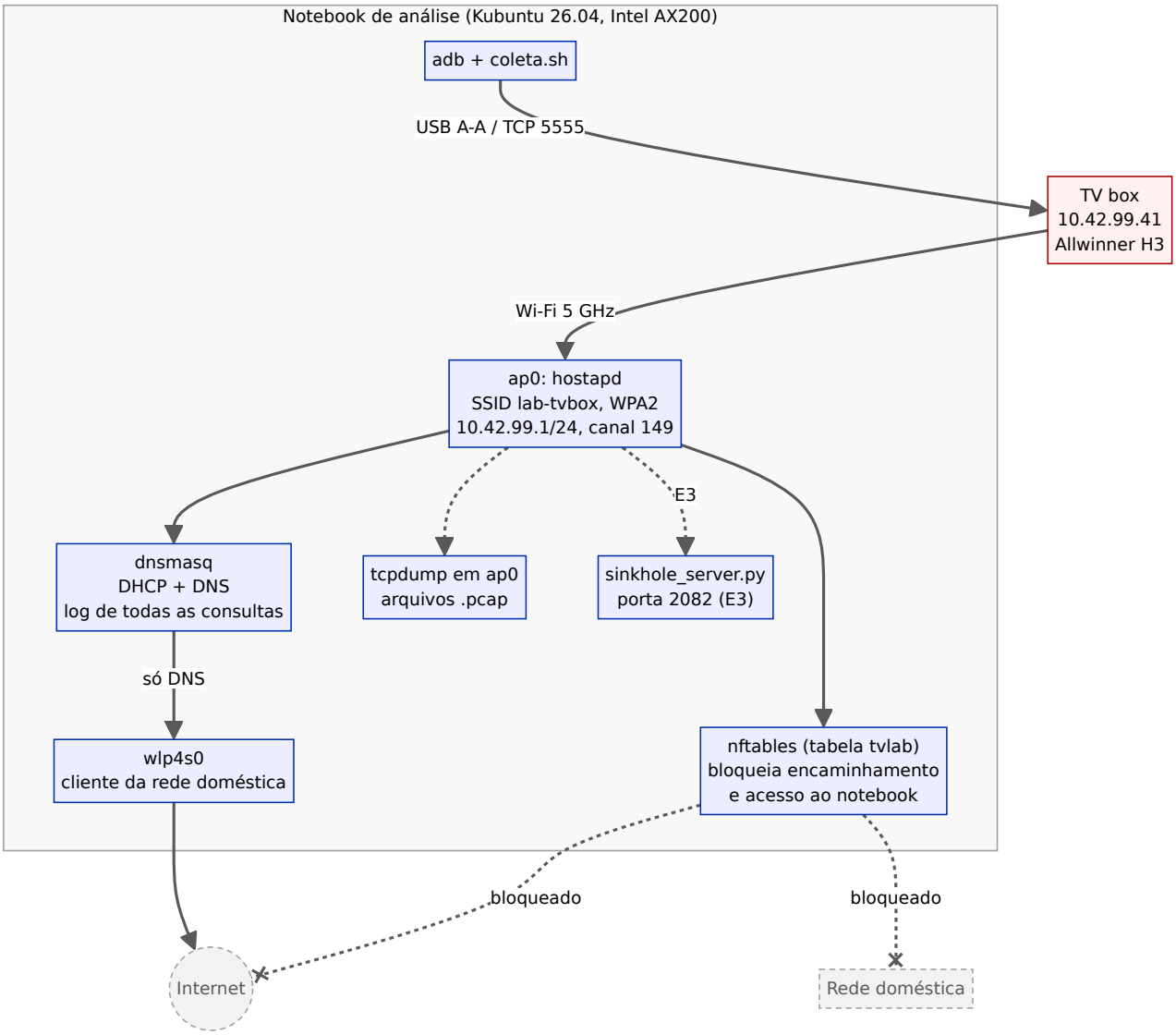


Figura 1. Diagrama do ambiente de teste.

2.1: Regras de isolamento

O script `lab/hotspot.sh` cria a tabela `inet tvlab` no nftables com as seguintes regras para tráfego vindo de `ap0`:

Cadeia	Regra	Efeito
forward	destino em 10/8, 172.16/12, 192.168/16, 169.254/16, 100.64/10	descarta: a box não alcança a rede doméstica
forward	qualquer IPv6	descarta
forward	qualquer outro destino (modo sem internet)	descarta: as tentativas aparecem como SYN sem resposta
input	UDP 53 e 67, TCP 53, conexões estabelecidas	aceita: DHCP e DNS do notebook
input	demais portas	descarta (no experimento 3 foi aberta a porta 2082)

O DNS foi mantido funcional de propósito. O dnsmasq resolve os nomes usando o servidor 1.1.1.1. Assim a box recebe o endereço IP real de cada domínio e tenta se conectar a ele. A conexão é bloqueada, mas o destino fica registrado no log DNS e na captura. A box não recebeu acesso à internet em nenhum momento. A decisão foi tomada porque a unidade é de origem desconhecida e possui aplicativos com privilégio de sistema que contatam servidores de terceiros.

2.2: Acesso ao sistema

A unidade não tinha ADB pela rede habilitado e não oferecia a opção "Depuração por Wi-Fi". O acesso foi feito pela opção de fábrica "usb0 device mode enable", que coloca uma das portas USB-A em modo dispositivo, com um cabo USB A-A ligado ao notebook. Essa opção volta ao estado original quando o cabo é removido e às vezes após reinicialização. Depois da conexão USB foi executado `adb tcpip 5555`, que habilita o ADB pela rede até o próximo boot. Na última sessão o ADB foi usado direto pelo USB.

3: Caracterização da unidade

Dados coletados por `getprop`, `/proc` e `/sys/class/sunxi_info` às 10:27 (arquivos em `evidencias/etapa0/`).

Item	Valor declarado ao usuário	Valor observado
Versão do Android	11.1 (`ro.build.version.release`)	7.0 Nougat: `ro.build.version.sdk=24`, build `NRD91N`
Patch de segurança	-	2016-11-05
Memória RAM	8 GB (`MemTotal: 8388608 kB`)	valor incompatível com o SoC H3, que endereça no máximo 2 GB: o kernel informa um valor falso
SoC	-	Allwinner H3, `sun8iw7p1`, ARMv7 32 bits, 4 núcleos
Fabricante / modelo	Allwinner / TV BOX	placa `dolphin-fvd-p1`, chip ID `02c00181...04d3`
Kernel	-	Linux 4.4.55, compilado em 12/12/2020
Build	-	`Allwinner/dolphin_fvd_p1/dolphin-fvd-p1:7.0/NRD91N/20201214173318:eng/test-keys`, usuário `akrd6`, host `R740XD`
Tipo de build	-	`eng` com `test-keys` (chaves públicas de teste do AOSP)
Segurança do ADB	-	`ro.secure=0`, `ro.debuggable=1`, `ro.adb.secure=0`: shell com UID 0 sem confirmação
SELinux	-	Permissive
Root	-	SuperSU (`eu.chainfire.supersu`) instalado em `/system`, com `daemonsu` ativo desde o boot
USB no boot	-	com o cabo ligado na porta OTG, a box aparece por cerca de 3 s como `1f3a:efe8` (modo FEL do Allwinner, usado para gravação de firmware) antes de iniciar o Android
Rede	-	MAC Wi-Fi `a4:7d:9f:24:a0:9c`, IP `10.42.99.41` (DHCP)
Fuso e idioma	-	`America/New_York`, `en-US`

Há 94 pacotes instalados: 48 em `/system/priv-app`, 41 em `/system/app` e 4 em `/data/app` (Kodi, YouTube para TV, GetApps/FlyMarket e `com.qcode.ad_service_server`). O firmware mistura componentes de três fabricantes de SoC diferentes: Allwinner (`com.softwinner.*`), Amlogic (launcher `com.droidlogic.mboxlauncher`) e Rockchip (`com.rk_itvui.allapp`). Também há dois apps com nome de código de exemplo executando como system: `com.example.android.autopair` e `com.example.android.usbbt`. Esses dados indicam um firmware genérico montado a partir de partes de SDKs diferentes. Esta é uma inferência que o Grupo 1 pode confirmar.

4: Metodologia

4.1: Ferramentas

Ferramenta	Uso
hostapd 2.11, dnsmasq, nftables 1.1.6	ponto de acesso isolado, DHCP, DNS com log (`--log-queries=extra`) e firewall
tcpdump	captura de todo o tráfego da interface `ap0`
tshark 4.6.4, capinfos	extração de consultas DNS, SYNs por destino e hierarquia de protocolos
adb 34.0.5	`ps`, `netstat -anp`, `/proc/net/*`, `dumpsys`, `logcat`, `/data/system/packages.list`
`lab/coleta.sh`	snapshots periódicos somente leitura com hash SHA-256 de cada arquivo
`lab/sinkhole_server.py`	servidor TCP que registra a requisição e responde sempre `404` vazio
dig, crt.sh	consulta passiva ao DNS público e aos registros de certificados de `uota.xyz`

4.2: Correlação conexão-processo

Cada snapshot grava `netstat -anp`. Como o shell do ADB executa como root, a coluna PID/programa vem preenchida para todos os sockets. O PID é cruzado com `ps` para obter o nome completo do processo. O UID é cruzado com `/data/system/packages.list` para obter o pacote. O IP de destino é cruzado com as respostas do log do dnsmasq para obter o nome de domínio. Conexões que abrem e fecham entre dois snapshots não aparecem no `netstat`. Nesses casos aparecem apenas na captura e no log DNS, sem processo associado.

4.3: Experimentos

ID	Horário	Estado da box	Captura de rede	Coleta ADB
E1	10:30-10:40	ociosa, ligada desde ~09:54	`captura_ociosa_sem_internet.pcap` (5 859 pacotes, 599 s)	7 snapshots a cada 90 s
E1b	10:45-10:54	ociosa (repetição de E1)	perdida, ver seção 10; log DNS preservado	7 snapshots a cada 90 s
-	~11:24	reinicialização	sem captura; log DNS preservado	logcat desde o boot
E2	11:28-11:43	reinicialização às 11:28:31	`captura_boot_sem_internet.pcap` (6 876 pacotes, 899 s)	logcat desde o boot + 7 snapshots
E3	12:00-12:32	2 reinicializações com sinkhole ativo	log DNS + requisições do sinkhole	uptime
E4	12:35-12:44	uso por uma pessoa logo após boot	`captura_interacao_sem_internet.pcap` (1 707 pacotes, 598 s)	logcat + 9 snapshots a cada 60 s
E5	14:40-15:00	ociosa, religada pouco antes de 14:30	log DNS	`netstat -anp` filtrado para a porta 80, cerca de 3 vezes por segundo
E6	15:59-16:19	logo após boot	`captura_ntp_sem_internet.pcap` (2 248 pacotes, 1 200 s)	sockets UDP via `netstat -anp`, cerca de 3 vezes por segundo (leitura até 16:00:19)

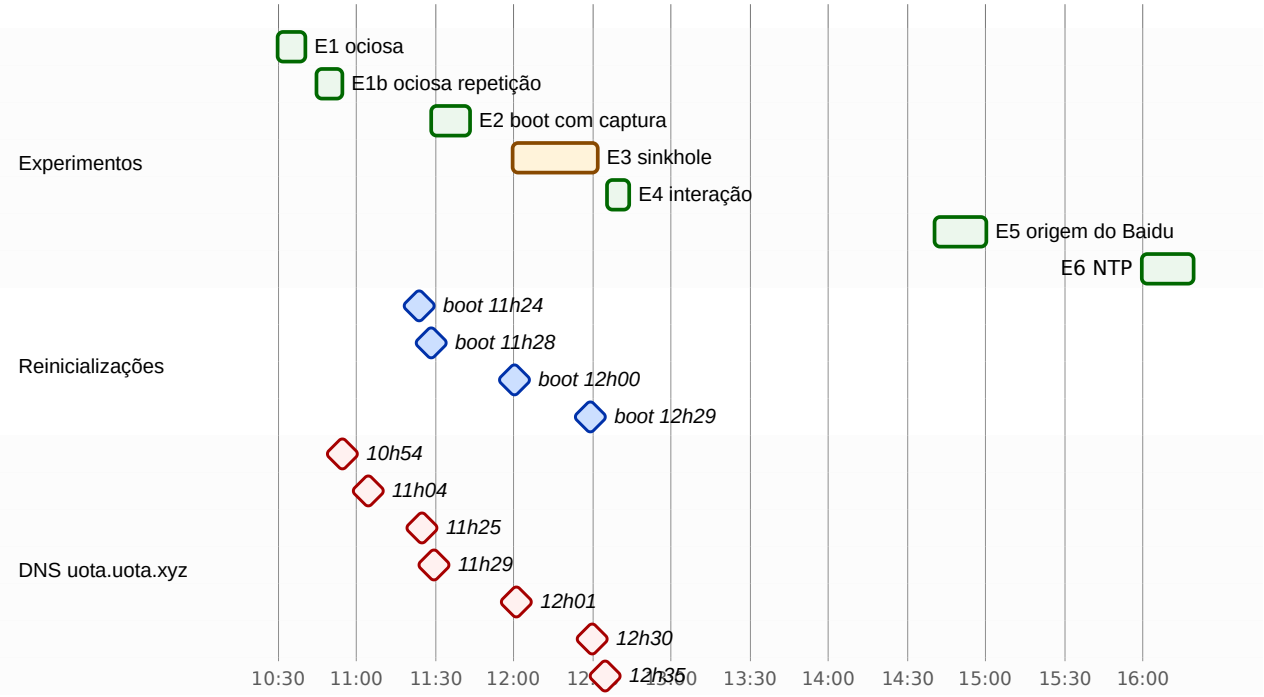


Figura 2. Linha do tempo dos experimentos, das reinicializações e das consultas DNS a `uota.uota.xyz`.

4.4: Critério de classificação

Foi usada a classificação da seção 11 da proposta: **Necessário** **Dispensável** **Desconhecido** **Potencialmente suspeito** **Vulnerável/mal configurado** e Malicioso comprovado. Nenhum item recebeu a última classificação. Conclusões que são inferências estão marcadas com a palavra "inferência".

5: Resultados por experimento

5.1: E1 e E1b: box ociosa

Com a box parada na tela inicial foram registradas tentativas de conexão destes processos:

- `com.google.android.gms` e `com.google.android.gms.persistent` (UID u0_a12): `android.clients.google.com`, `mtalk.google.com`, `play.googleapis.com`, porta 443 TCP e QUIC;
- `com.google.android.youtube.tv.recommendations` (u0_a64): servidores do Google, porta 443, com a maior frequência de SYNs da rodada (36 sockets em SYN_SENT nos snapshots);
- `com.android.vending` (Play Store): `android.clients.google.com`, porta 443;
- `com.uv.droid.uota` (PID 5452) e `com.uv.droid.uotaback` (PID 10321), ambos UID 1000: `www.google.com`, porta 80 (HTTP);
- sem processo associado nos snapshots: `www.baidu.com` (103.235.46.96 e 103.235.47.188), porta 80. O processo foi identificado depois, no experimento E5 (seção 5.5);
- servidores NTP (`*.pool.ntp.org` e IPs sem consulta DNS), UDP 123. A atribuição foi revista no experimento E6 (seção 5.6).

Na rodada E1 não houve consulta a `uota.uota.xyz`. Durante E1b os mesmos processos voltaram a aparecer. Os dois processos UOTA mantiveram o mesmo PID durante as duas rodadas, ou seja, ficaram em execução contínua. A box consultou `uota.uota.xyz` às 10:54:55, logo depois do fim da coleta de E1b. A consulta se repetiu às 11:04:55. Nesse momento a box estava ligada havia cerca de uma hora.

Portas abertas na box

Porta	Processo	Interpretação	Classificação
5555/tcp	adbd	aberta pela equipe com `adb tcpip`; ao ser aberta, aceita conexão com shell root sem autenticação	**Vulnerável**
6466/tcp, 6467/tcp	`com.google.android.remote.tv.services`	serviço de controle remoto do Android TV (pareamento e comandos)	**Dispensável**
5353/udp	mdnsd	anúncio mDNS `TV BOX._androidtvremote._tcp.local`	**Dispensável**
5037/tcp (127.0.0.1)	adbd	escuta apenas local	**Necessário** (análise)

5.2: E2: reinicialização

A captura foi iniciada antes do desligamento. A box foi religada às 11:28:31 e o ADB voltou às 11:29:00. A ordem das primeiras consultas DNS (tempo em segundos a partir do início da captura) foi:

t (s)	Nome	t (s)	Nome
72	2.android.pool.ntp.org	117	android.clients.google.com
77	0.pool.ntp.org	152	alt5-mtalk.google.com
78	mtalk.google.com	159	time.google.com
79	**uota.uota.xyz**	169	www.baidu.com
81	www.googleapis.com	199	www.google.com

82	youtubei.googleapis.com	426	redirector.gvt1.com

`uota.uota.xyz` resolveu para 104.21.33.88 e 172.67.160.182, endereços da Cloudflare. Entre 79 s e 88 s a box enviou 16 SYNs para 104.21.33.88 na porta 2082. O `netstat` associou a conexão ao PID 5219, que é `com.uv.droid.uotaback`. Depois do primeiro minuto não houve novas tentativas até o fim da captura (15 min).

O logcat do boot mostra os receptores de inicialização (mecanismos de persistência) acionados. Entre eles:

- `com.uv.droid.uota/.receiver.LoaderReceiver`
- `com.uv.droid.uotaback/.receiver.LoaderReceiver`
- `eu.chainfire.supersu/.BootCompleteReceiver`
- `com.rk_itvui.allapp/.StbBootReceiver`
- `com.example.android.autopair/.BootCompleteReceiver` e `com.example.android.usbbt/.BootCompleteReceiver`
- `com.softwinner.dragonbox/.StorageReceiver`

Os apps UOTA não escrevem nenhuma linha no logcat.

O relógio da box volta para 31/12, 19:00, a cada boot. A unidade não tem relógio com bateria e o NTP está bloqueado. Por isso os horários do logcat não correspondem ao horário real. Neste relatório todos os horários são do notebook.

5.3: E3: sinkhole de uota.uota.xyz

Antes do experimento foram feitas consultas passivas. O domínio `uota.xyz` usa servidores de nomes da Cloudflare (`kallie` e `grant.ns.cloudflare.com`) e o subdomínio `uota.uota.xyz` aponta para a CDN da Cloudflare. O endereço real do servidor de origem não é visível. Os registros de transparência de certificados (crt.sh) mostram certificados para `uota.xyz` e `*.uota.xyz` desde 12/03/2019 até 20/09/2026. Não foi encontrado relatório público de segurança sobre o domínio ou sobre o pacote `com.uv.droid.uota`.

No sinkhole, o dnsmasq passou a responder `*.uota.xyz` com o IP do notebook. Na porta 2082 do notebook ficou um servidor que grava a requisição e responde `HTTP 404` sem corpo. A box continuou sem internet, o servidor real não foi contatado e nenhum conteúdo foi entregue à box.

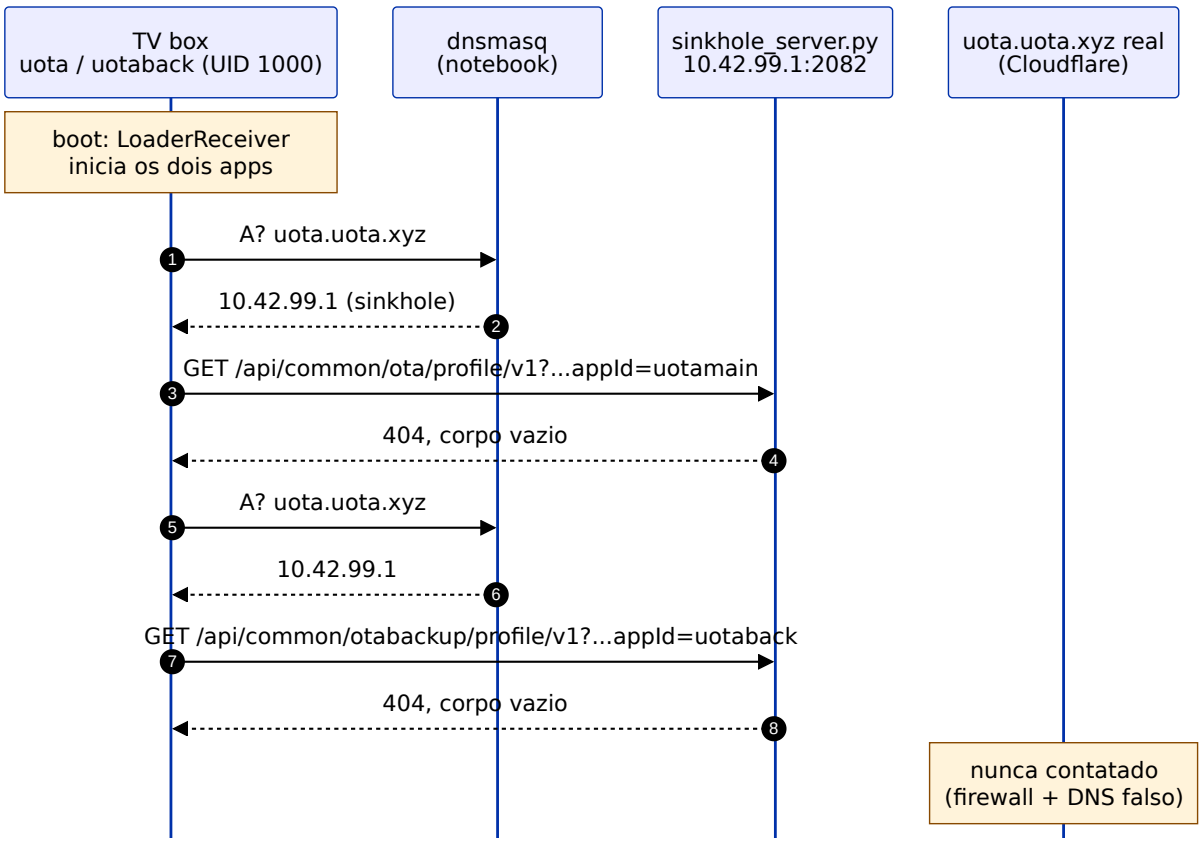


Figura 3. Sequência observada no experimento E3.

Nas duas reinicializações (requisições às 12:01:16, 12:01:18, 12:30:13 e 12:30:15) cada app enviou uma requisição. O uptime de 1 minuto foi confirmado por ADB logo depois. As duas requisições de cada boot têm cerca de 2 segundos de diferença. Elas usam HTTP/1.1 sem TLS, `User-Agent: okhttp/3.8.1` e nenhum cabeçalho de autenticação. Os parâmetros enviados foram:

Parâmetro	Valor	Natureza
`wifiMac`	`a4:7d:9f:24:a0:9c`	identificador de hardware persistente
`cpuSerial`	`143078f2080c442704d3`	identificador de hardware persistente
`androidId`	`696298afbf9fba22`	identificador do aparelho (muda apenas com reset de fábrica)
`did`	`a82009eca3c6`	identificador próprio do serviço
`fingerprint`, `brand`, `model`, `deviceName`, `os`, `cpuHardwareModel`	build, `Allwinner`, `TV BOX`, `dolphin-fvd-p1`, `11.1`, `sun8iw7`	identificação do firmware
`firmwareVersion`, `firmwareName`, `firmwareClient`	`1607938396` (14/12/2020 em Unix time), `SoftFirmware`, `unknownClient`	versão do firmware
`appId`, `appVer`, `clientPackageName`, `isSystemApp`	`uotamain`/10003 e `uotaback`/10002, pacote, `true`	identificação do cliente
`launcherPackageName` (só `uota`)	`com.droidlogic.mboxlauncher:24;com.android.settings:24`	apps instalados no aparelho

5.4: E4: uso por uma pessoa

Logo após o boot das 12:30 a box foi usada com teclado e mouse. Os nomes das atividades vêm do log `ActivityManager: START`:

Ação	Atividade registrada	Rede
Kodi	`org.xbmc.kodi/.Splash`, `.Main`	nenhum destino novo
YouTube	`com.google.android.youtube.tv/...ShellActivity`	`www.youtube.com`:443. O app ficou preso na tela de carregamento e só saiu com `input keyevent KEYCODE_HOME` pelo ADB
"GetApps"	`com.simple.appstore.flymarket/.activity.WelcomeActivity`, `.MainActivity`; em seguida `GrantPermissionsActivity`	`appstore.bb20.site` (104.219.250.37, 2.59.170.20) porta 2086, 32 SYNs; `android.bugly.qq.com` (14.22.7.140, 14.22.7.199, 119.147.179.152) porta 80, ao menos 10 SYNs
Play Store	`com.android.vending/.AssetBrowserActivity`, fluxo `AccountIntroActivity`	servidores do Google, porta 443
Configurações	`com.android.settings/.Settings`	nenhum destino novo

O ícone "GetApps" abre o pacote `com.simple.appstore.flymarket`. O nome exibido ao usuário é diferente do nome do pacote. Ao ser aberta pela primeira vez a loja solicitou permissão de armazenamento e ela foi concedida. A permissão foi revogada às 13:16 (seção 11). O `android.bugly.qq.com` pertence ao Tencent Bugly, SDK de relatório de falhas. A porta 2086 é a porta HTTP sem TLS alternativa usada pela Cloudflare. Os IPs de `appstore.bb20.site` não pertencem à Cloudflare, por isso o uso de HTTP sem TLS nessa porta é uma inferência. Como a conexão foi bloqueada, não há conteúdo para confirmar.

O aplicativo `com.qcode.ad_service_server` (versão 1.0.7777.976, em `/data/app`) não tem ícone no launcher. Ele não apareceu em nenhum `ps`, em nenhuma linha de log e em nenhuma conexão em E1, E1b, E2 e E4.

5.5: E5: origem da consulta ao Baidu

Nos experimentos E1 a E4 a conexão para `www.baidu.com` não apareceu em nenhum snapshot. A conexão é curta e os snapshots tinham intervalo de 60 s a 90 s. Como a consulta DNS se repete em intervalos previsíveis, foi feito um experimento específico: o script `lab/caca_porta80.sh` executou `netstat -anp` na box cerca de três vezes por segundo durante 20 minutos e registrou toda conexão para a porta 80, com horário do notebook. O script só lê dados e se reconecta quando o USB oscila.

Consulta DNS	Conexão observada	Porta local	Processos associados (UID 1000)
14:44:25	14:44:41 → 103.235.46.96:80, SYN_SENT	4458	`com.uv.droid.uota` (PID 4520), `com.uv.droid.uotaback` (PID 4670)
14:50:02	14:50 → 103.235.46.96:80, SYN_SENT	4467	os mesmos dois PIDs
14:57:32	14:57 → 103.235.46.96:80, SYN_SENT	4474	os mesmos dois PIDs

Nos três eventos houve um único socket por tentativa. O `netstat` atribuiu esse socket ora ao PID do `uota`, ora ao do `uotaback`, em número quase igual de leituras. Os dois processos executam com o mesmo UID e a ferramenta não permite decidir qual deles abriu o socket. A conclusão segura é que a conexão pertence ao par UOTA. Com isso, os mesmos apps que consultam `uota.uota.xyz` também acessam `www.google.com` e `www.baidu.com` por HTTP. A finalidade foi esclarecida pela análise estática do Grupo 1: o código (`WebTimeProvider`, nos dois apps) lê o cabeçalho HTTP `Date` dessas respostas como fonte de hora. A inferência inicial deste relatório, de teste de conectividade, estava errada (seção 9). O intervalo entre consultas é de 7 min 30 s na maior parte do registro, com variações (14:44:25 → 14:50:02).

5.6: E6: auditoria de cobertura e NTP com IPs fixos

Depois do E5 foi feita uma auditoria para verificar se restava algum destino sem processo associado. Todos os destinos das capturas (SYNs TCP e datagramas UDP) foram cruzados com os sockets registrados pelo `netstat` em todas as coletas e com os nomes respondidos pelo dnsmasq. Os destinos TCP sem socket registrado pertencem aos mesmos domínios de processos já identificados (Google, Tencent Bugly, `uota.uota.xyz`). A exceção foram pedidos NTP para seis endereços que a box nunca consultou no DNS:

IP de destino	DNS reverso
176.9.118.9	`static.9.118.9.176.clients.your-server.de` (Hetzner, Alemanha)
61.216.153.106	`61-216-153-106.hinet-ip.hinet.net` (HiNet, Taiwan)
203.135.184.123	`leontp.ccgs.wa.edu.au` (escola na Austrália)
108.59.2.24	`ntp.wdc2.us.leaseweb.net` (Leaseweb, EUA)
164.73.227.4	sem nome reverso
201.217.3.86	`ntp2.copaco.com.py` (Copaco, Paraguai)

Esses pedidos aparecem nas três capturas anteriores, sempre na mesma ordem. Para identificar a origem, a box foi religada às ~15:58 com captura de rede ativa e o script `lab/caca_ntp.sh` registrou todos os sockets UDP com o PID dono. A porta de origem de cada pacote na captura foi cruzada com a porta local dos sockets. O USB caiu às 16:00:19 e não voltou durante o experimento. Por isso a leitura de sockets cobre apenas o primeiro ciclo. A captura cobre os 20 minutos.

Hora	Destino	Origem do nome	Portas de origem	Processos
15:59:36	200.189.40.8	`0.pool.ntp.org`	57863, 33825	PID 3904, PID 3974
15:59:41	176.9.118.9	IP fixo	51183, 35870	PID 3904, PID 3974
15:59:46	200.160.0.8	`1.pool.ntp.org`	57369, 47176	PID 3904, PID 3974
15:59:51	61.216.153.106	IP fixo	36739, 52370	PID 3904, PID 3974
15:59:56	200.25.72.83	`2.pool.ntp.org`	33565, 53870	PID 3904, PID 3974
16:00:01	203.135.184.123	IP fixo	58767, 60115	PID 3904, PID 3974
16:00:06	192.36.143.130	`*.pool.ntp.org`	55061, 56805	PID 3904, PID 3974
16:00:11	108.59.2.24	IP fixo	39426, 47502	PID 3904, PID 3974

Os 16 pacotes do primeiro ciclo têm dono. Os PIDs 3904 e 3974 aparecem no `netstat` como `com.uv.droid.uo…` (nome truncado pela ferramenta), ou seja, são o par `uota`/`uotaback`. Cada app

percorre a mesma lista: um nome do `pool.ntp.org`, depois um IP fixo, de novo um nome e assim por diante, com 5 s entre servidores e 2 s entre os dois apps. O ciclo se repetiu às 16:06, 16:12 e 16:19, a cada 6 a 7 minutos. Os 118 pacotes NTP da captura têm 48 bytes de conteúdo, versão 3, modo cliente, com todos os campos zerados exceto o horário de envio. Ou seja, são pedidos de hora padrão sem dados adicionais.

Interpretação: o UOTA faz a própria sincronização de hora com uma lista gravada no código. O Grupo 1 confirmou a lista (`SntpTimeProvider`): são 18 entradas, com os seis IPs acima, nomes do `pool.ntp.org`, `time.google.com` e outros três IPs (200.189.40.8, 200.192.232.8, 216.239.35.12). O horário obtido é usado em caches e temporizações do próprio app. Como `time.google.com` também está nessa lista, apenas `2.android.pool.ntp.org`, servidor padrão do Android, segue atribuído ao sistema por inferência.

6: Tabela consolidada: processo, destino, protocolo e comportamento

Processo / pacote (UID)	Destino	Protocolo / porta	Comportamento observado	Exp.	Classificação
`com.uv.droid.uota` (1000)	`uota.uota.xyz` (Cloudflare)	HTTP / 2082	requisição de perfil OTA com identificadores do aparelho em texto claro; após o boot e em horários sem boot	E1b, E2, E3	**Vulnerável** **Suspeito**
`com.uv.droid.uotaback` (1000)	`uota.uota.xyz` (Cloudflare)	HTTP / 2082	mesma requisição em `/otabackup/`; 16 SYNs no 1º minuto do boot	E2, E3	**Vulnerável** **Suspeito**
`com.uv.droid.uota`, `.uotaback` (1000)	`www.google.com`	TCP / 80	SYNs repetidos com a box ociosa; fonte de hora pelo cabeçalho HTTP `Date` (Grupo 1)	E1, E1b	**Suspeito**
`com.uv.droid.uota`, `.uotaback` (1000)	`www.baidu.com` (103.235.46.96, 103.235.47.188)	TCP / 80	tentativa a cada ~7 min 30 s, inclusive com a box ociosa; processo identificado em E5 (3 de 3 eventos); fonte de hora pelo cabeçalho HTTP `Date` (Grupo 1)	todos, E5	**Suspeito**
`com.simple.appstore.flymarket` ("GetApps")	`appstore.bb20.site`	TCP / 2086	32 SYNs ao abrir a loja	E4	**Suspeito**
`com.simple.appstore.flymarket` (SDK Bugly)	`android.bugly.qq.com` (China)	HTTP / 80	ao menos 10 SYNs ao abrir a loja	E4	**Dispensável**
`com.google.android.gms`, `.persistent` (u0_a12)	`android.clients.google.com`, `mtalk.google.com`, `*.googleapis.com`, 108.177.123.188	TLS / 443, 5228; QUIC	tentativas contínuas	todos	**Necessário**
`com.android.vending` (u0_a22)	`android.clients.google.com`	TLS / 443	tentativas no boot e ao abrir	todos	**Necessário**
`com.google.android.youtube.tv`, `.recommendations` (u0_a64)	`youtubei.googleapis.com`, `www.youtube.com`, `redirector.gvt1.com`	TLS / 443	maior volume de SYNs com a box ociosa	todos	**Dispensável**
`com.uv.droid.uota`, `.uotaback` (1000)	`0-3.pool.ntp.org` e 6 IPs fixos (seção 5.6)	NTP / UDP 123	pedidos de hora padrão a cada 6 a 7 min, lista própria embutida	todos, E6	**Suspeito**

sistema Android (inferência)	`2.android.pool.ntp.org`	NTP / UDP 123	sincronização de hora	todos	**Necessário**
`mdnsd`, `remote.tv.services`	224.0.0.251 (rede local)	mDNS / UDP 5353	anúncio do serviço de controle remoto	todos	**Dispensável**
`org.xbmc.kodi`	nenhum	-	sem tráfego ao abrir	E4	**Dispensável**
`com.qcode.ad_service_server`	nenhum	-	não executou	todos	**Desconhecido**

Volume aproximado

A seção 9.3 da proposta pede o volume aproximado das comunicações. Como a box não teve acesso à internet, nenhuma conexão externa foi concluída e não há volume de dados trocados com os servidores. O que pode ser medido é o volume que a box enviou ao tentar se conectar: pacotes SYN, datagramas QUIC e NTP e suas retransmissões. A tabela soma as quatro capturas (E1, E2, E4 e E6), que totalizam cerca de 55 minutos, sem o tráfego ADB e o tráfego local (DHCP, DNS para o notebook e mDNS).

Destino (agrupado)	Processo	Pacotes	Bytes
Google (GMS, Play Store, YouTube)	apps do Google	3 512	863 366
`www.google.com`, porta 80	par UOTA	548	40 552
NTP: `pool.ntp.org` e `time.google.com`	par UOTA (e sistema, para `2.android.pool.ntp.org`)	176	15 840
`www.baidu.com`, porta 80	par UOTA	144	10 656
NTP: seis IPs fixos	par UOTA	101	9 090
`appstore.bb20.site`, porta 2086	GetApps	64	4 736
`uota.uota.xyz`, porta 2082	par UOTA	32	2 368
`android.bugly.qq.com`, porta 80	GetApps (SDK Bugly)	30	2 220
Total		**4 607**	**948 828**

Esses números medem a insistência de cada app e não o tamanho real da comunicação. Os valores do Google são altos porque os pacotes iniciais do QUIC têm cerca de 1 200 bytes e são retransmitidos. O único conteúdo de aplicação observado foi a requisição HTTP do UOTA no sinkhole (seção 5.3), com 604 bytes (uotaback) e 669 bytes (uota) por requisição.

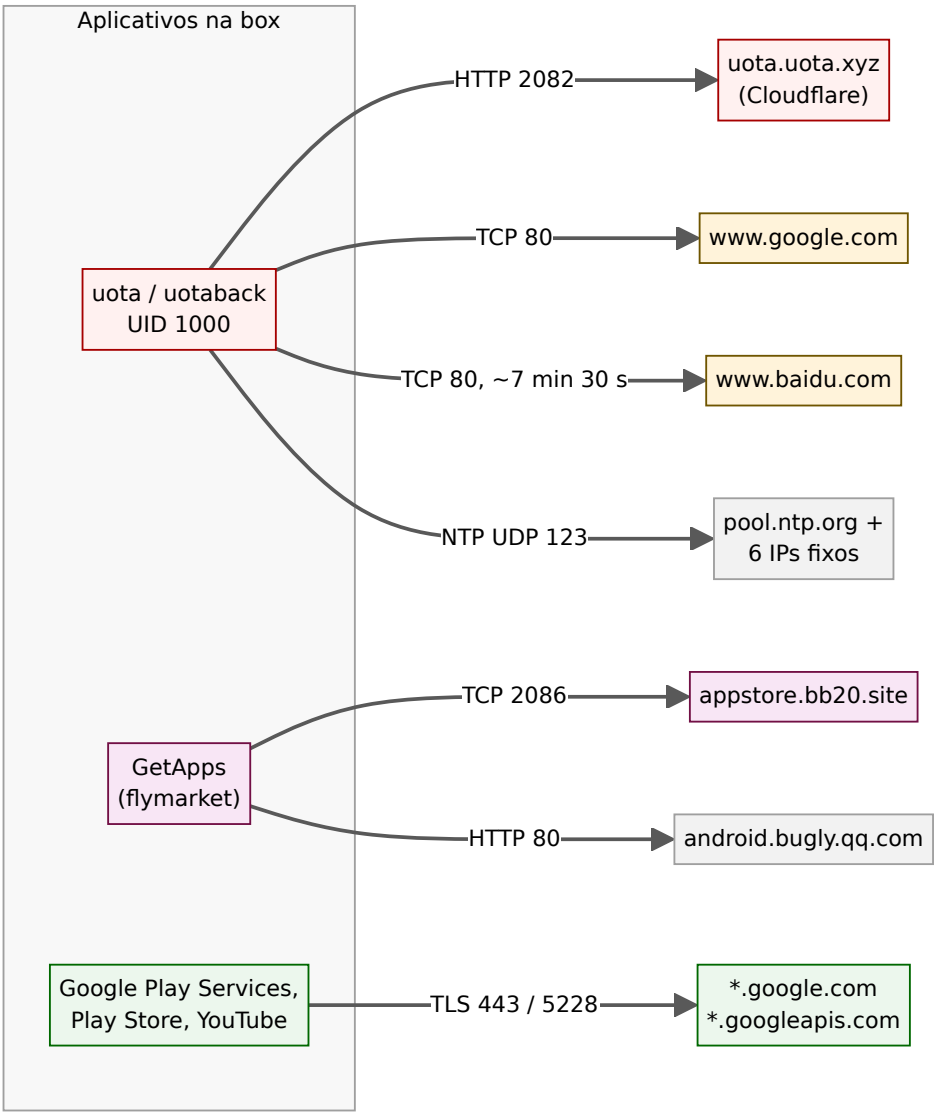


Figura 4. Destinos externos por aplicativo. As cores seguem a classificação da tabela.

7: Comportamentos explicados e não explicados

7.1: Explicados

- Tráfego do Google Play Services, da Play Store e do YouTube para domínios do Google, que é o comportamento padrão desses apps.
- Consultas NTP do sistema (`2.android.pool.ntp.org`) após o boot. O relógio da box reinicia a cada boot e precisa de sincronização.
- Pedidos NTP do par UOTA para `pool.ntp.org` e para seis IPs fixos no código (E6). São pedidos de hora padrão, sem dados adicionais.
- Anúncio mDNS e portas 6466/6467 do serviço de controle remoto do Android TV.
- Requisição de perfil OTA dos apps UOTA após o boot. Verificar atualizações na inicialização é o comportamento esperado de um atualizador. O que foi classificado como problema é a forma: texto claro e identificadores persistentes.
- Contato com o Tencent Bugly pela loja GetApps, que é o comportamento esperado de um SDK de relatório de falhas embutido.
- Conexões HTTP do par UOTA com `www.google.com` e `www.baidu.com` (processo identificado no E5). Segundo o Grupo 1, o app usa o cabeçalho HTTP `Date` da resposta como fonte de hora.

7.2: Não explicados

- **Intervalo das consultas a `uota.uota.xyz` fora do boot.** Houve consultas às 10:54:55 e 11:04:55 (box ligada havia ~1 h) e às 12:35:11 (5 min após um boot). Entre 11:29 e 12:00 não houve consulta. O critério de agendamento não foi determinado.
- **Gatilho de `com.qcode.ad_service_server`.** O app não executou em nenhum experimento e não tem ícone.

8: Matriz de achados

ID	Achado	Evidência	Classificação	Grau
A1	ADB com shell root sem autenticação quando a porta 5555 é aberta (`ro.adb.secure=0`, `ro.secure=0`)	`etapa0/getprop.txt`; `id` retornou `uid=0(root)`	**Vulnerável**	demonstrado
A2	Versão do Android e quantidade de RAM falsificadas (11.1 e 8 GB; real 7.0 e no máximo 2 GB)	`getprop`, `/proc/meminfo`, `sunxi_info`	**Mal configurado**	demonstrado
A3	Firmware `eng`/`test-keys`, patch de 2016, SELinux permissivo, SuperSU de fábrica	`getprop`, `getenforce`, `ps`	**Vulnerável**	demonstrado
A4	Apps UOTA (UID 1000, assinados com a chave de plataforma) enviam MAC, serial da CPU e Android ID em HTTP sem TLS para `uota.uota.xyz`	`sinkhole/req_*.bin`; 4 requisições em 2 boots	**Vulnerável**	demonstrado
A5	Canal de atualização de app com privilégio de sistema sem TLS e sem autenticação: um intermediário poderia responder com conteúdo próprio	A4 + UID 1000	**Suspeito**	inferência
A6	Servidor de atualização escondido atrás da Cloudflare, em domínio `.xyz` sem relação com fabricante identificável	DNS público, crt.sh	**Suspeito**	demonstrado
A7	Loja alternativa com nome exibido diferente do pacote, backend `appstore.bb20.site:2086` e SDK chinês de telemetria	`captura_interacao_sem_internet.pcap`, log DNS	**Suspeito**	demonstrado
A8	Par UOTA acessa `www.baidu.com` e `www.google.com` por HTTP a cada ~7 min 30 s para obter a hora	`dns/dnsmasq.log`, `dinamica/caca_porta80_*/`; código: Grupo 1	**Suspeito**	processo e finalidade demonstrados
A11	Par UOTA usa lista própria de servidores NTP com seis IPs fixos, entre eles um servidor de escola australiana	`captura_ntp_sem_internet.pcap`, `dinamica/caca_ntp_*/`	**Suspeito**	demonstrado
A12	Modo FEL do Allwinner exposto na porta OTG durante o boot	`journalctl` do notebook, 15:58:39	**Desconhecido**	observado uma vez
A9	`com.qcode.ad_service_server` não executou; o Grupo 1 encontrou o pacote no estado `stopped`, que impede a entrega do `BOOT_COMPLETED`. O código tem uma rotina de boot que apaga arquivos acima de 10 MiB no armazenamento externo	todos os snapshots e logs; código: Grupo 1	**Suspeito**	não execução explicada por hipótese; exclusão é capacidade, sem execução observada

A13	O serviço root `qw`, iniciado pelo `preinstall.sh` (Grupo 1), mantém o socket `/dev/com.koushikdutta.superuser.daemon/server`: é o daemon de root do Superuser com outro nome. Não abriu nenhum socket de rede	`dinamica/*/snap_*/netstat.txt`, `ps.txt`	**Suspeito**	função identificada pelo nome do socket (inferência forte)
A10	Firmware com componentes de Allwinner, Amlogic e Rockchip e apps `com.example.*` como system	`etapa0/pacotes.txt`, `packages.list`	**Desconhecido**	inferência

9: Correlação com os Grupos 1 e 3

O Grupo 1 (Maria Rita e Pedro Gabryel) entregou o relatório "Auditoria estática de uma TV box Android" e o arquivo `RESPOSTAS_GRUPO2.md` em 13 de outubro de 2026, analisando a mesma unidade (serial `14f2782704d344000052`). A tabela cruza cada achado deste relatório com o que o Grupo 1 encontrou no código. O Grupo 3 (Francisco Tiago Freitas Lima, Olimpio de Carvalho Andrade e Wellyson dos Santos Silva) entregou o relatório de hardening na mesma data. Ele usou outra unidade (`G3-65f278220a133c00006d`), com a mesma placa e a mesma fingerprint de firmware; os resultados são comparáveis, mas não são da mesma box. A seção 9.1 trata do hardening.

Achado	Grupo 2: observado em execução	Grupo 1: encontrado no código	Conclusão conjunta
A4/A5 UOTA	requisição HTTP de perfil com MAC, serial da CPU e Android ID (sinkhole, 2 de 2 boots)	perfil → `POST update/v1` → download da URL recebida → MD5 recebido pelo mesmo canal → `pm install -r` silencioso no método SILENT e para os próprios UOTA; validação TLS desligada (`HostnameVerifier` sempre verdadeiro); modo `command.do` que decifra comandos com chave AES fixa e os passa ao `sh`	A5 deixa de ser inferência: quem controlar o canal HTTP pode entregar um APK com MD5 correspondente. No sinkhole, que respondia 404, o app nunca chamou `update/v1`, o que é compatível com o código: sem perfil válido o fluxo não avança. Vulnerável (G1-03, G1-04); o modo de comandos é potencialmente suspeito (G1-05). Nenhum conteúdo hostil foi observado
A8 Google/Baidu	conexão porta 80 do par UOTA em 3 de 3 eventos (E5)	`WebTimeProvider` lê o cabeçalho HTTP `Date` como fonte de hora	a inferência deste relatório (teste de conectividade) estava errada; é fonte de hora sem autenticação. Classificação revista: este relatório classificava como dispensável; após discussão com o Grupo 1, passou a potencialmente inseguro, porque o código usa essa hora sem autenticação em caches e temporização
A11 NTP com IPs fixos	16 de 16 pacotes do primeiro ciclo com dono no par UOTA (E6)	lista de 18 fontes em `SntpTimeProvider`, tentadas em ordem até uma responder	confirmado; explica a sequência ordenada e a repetição vistas na captura. Classificação revista da mesma forma que o item anterior: de dispensável para potencialmente inseguro
Dois UOTA juntos	os dois processos sempre em execução; contato com `uota.uota.xyz` após conectar à rede	`uotaback` atualiza o principal apenas se ele estiver instalado; `LoaderReceiver` inicia o serviço quando há rede	manutenção cruzada; não é rotina de reinstalação. Explica por que o contato acontece logo após o Wi-Fi conectar

A9 qcode	não executou em boot, ociosidade ou uso	pacote em `stopped=true` / `notLaunched=true` na coleta de 12/10; receiver de boot com rotina que apaga arquivos acima de 10 MiB	nos 3 logs de boot do Grupo 2, os mesmos 7 receptores de boot iniciaram e o qcode não iniciou em nenhum. Isso reforça a hipótese do estado `stopped` também nos experimentos de 9/10. A exclusão é capacidade não observada (G1-06)
A13 `qw`	root, socket `/dev/com.koushikdutta.superuser.daemon/server`, nenhum socket de rede	serviço root iniciado no fim do `preinstall.sh`; função não determinada (G1-08)	a evidência dinâmica indica o daemon do Superuser renomeado. Sem atividade de rede observada
`appmetrices.uota.xyz:2095`, `vxmfc.com`	nenhuma consulta DNS a nenhum dos dois em todo o registro de 9/10 (10:11 a 19:32)	endpoint de métricas configurado no UOTA principal; `vxmfc.com` só como cabeçalho Referer do downloader	configurado, mas não observado nos experimentos (resultado negativo)
A7 GetApps	`appstore.bb20.site:2086` e Bugly ao abrir a loja	código protegido por empacotador nativo (jiagu); sem permissão de instalar pacotes	inconclusivo; a construção do endereço não foi reconstruída
A12 FEL	`1f3a:efe8` por ~3 s no boot	não testado	aberto

9.1: Medidas do Grupo 3 e efeito observado

Achado	Medida do Grupo 3	Resultado registrado pelo Grupo 3	Conclusão conjunta
A4/A5 UOTA	`pm disable-user` em `uota` e `uotaback`	desativação persistiu após reboot; `OtaService` e os dois processos ausentes	sem os processos, não há origem para o tráfego que o Grupo 2 atribuiu ao par UOTA (porta 2082, Google/Baidu na porta 80, NTP com IPs fixos). É uma conclusão por inferência: o tráfego depois do hardening não foi capturado
Portas 6466/6467 e mDNS	`pm disable-user` no serviço de controle remoto (2ª tentativa)	apenas `127.0.0.1:5037` em escuta; `mdnsd` encerrado; o processo persistente do serviço continua vivo, sem portas	exposição de rede eliminada, como o Grupo 2 havia sugerido
A9 qcode, A7 GetApps	`pm hide`, porque o `pm disable-user` era desfeito no boot	ocultação persistente após reboot	ver o caso conjunto na seção 9.2
Fontes desconhecidas	`install_non_market_apps` de 1 para 0	instalador gráfico bloqueia APK de teste; `adb install` continua funcionando	reduz a instalação pelo usuário; não afeta o caminho do UOTA, que usa `pm install`
A1/A3 ADB, root, build	não corrigidos (exigem regravar o firmware)	ADB sem escuta na rede na unidade do Grupo 3	risco residual; as três análises concordam

H1 bloqueio de DNS	não aplicado	a unidade ficou sem rede durante o hardening	sem resultado

Na matriz de funcionalidades do Grupo 3, imagem HDMI, áudio e reprodução de vídeo seguiram funcionando após as medidas. Wi-Fi, Ethernet e modo kiosk não foram testados. A restauração a partir das cópias das partições não foi executada.

9.2: Um caso visto pelos três grupos

- **Grupo 3:** o `pm disable-user` no qcode e na GetApps era desfeito a cada reinicialização.
- **Grupo 1:** o `preinstall.sh`, executado como root no boot, só deveria reinstalar os apps de `/system/preinstall` na primeira inicialização, mas os arquivos-marca que controlam isso não existem. A pré-instalação roda em todo boot.
- **Grupo 2:** a última linha desse script inicia o `qw`, que nas nossas coletas mantém o socket do daemon do Superuser e roda como root.

Juntas, as três observações explicam por que a desativação comum não persiste e mostram que o mesmo script mantém um daemon de root ativo. A causa exata da reversão não foi confirmada pelo Grupo 3; a explicação acima é a hipótese mais compatível com os três relatórios.

9.3: Pendências da correlação

- Captura de tráfego após o hardening, para confirmar diretamente que os destinos do UOTA desaparecem.
- Conteúdo que o servidor real enviaria ao UOTA: depende de acesso à internet, que não foi liberado.
- Camada nativa da GetApps, teste do modo FEL, confirmação da função do `qw` pelo binário e teste de restauração das partições.

9.4: Recomendações conjuntas

- Manter o UOTA desativado e bloquear `*.uota.xyz` no DNS da rede como segunda barreira.
- Usar `pm hide` nos apps reinstalados pelo `preinstall.sh`, ou corrigir o script, que exige alterar `/system`.
- Desabilitar o ADB na configuração final; o bloqueio de fontes desconhecidas não impede `adb install`.
- Avaliar a remoção do SuperSU e do daemon `qw`.
- Por causa de A2 e A3 (Android 7 com patch de 2016, build `eng`), buscar firmware confiável em vez de apenas endurecer este.
- Repetir hardening e medições em outras unidades, com captura de tráfego antes e depois.

10: Limitações e desvios de procedimento

Box sem internet. A decisão de não liberar a internet impede ver as respostas dos servidores, o que seria baixado e o comportamento posterior. Os resultados mostram para onde a box tenta ir e o que envia ao iniciar a conexão. Não mostram o que ela faria com a resposta. Pelo mesmo motivo o volume aproximado pedido na seção 9.3 da proposta foi medido como volume de tentativas enviadas pela box (seção 6), já que nenhuma comunicação externa foi concluída. A proposta exige isolamento em relação à rede institucional e não proíbe acesso à internet. Ficar sem internet foi uma decisão da equipe, justificada na seção 2.1.

- **Instrumentação da equipe.** A porta 5555 foi aberta pela equipe. O tráfego ADB aparece nas capturas (porta 5555) e foi desconsiderado. A conexão do ADB também aparece no `netstat`.
- **Logcat apagado no início de E1.** A primeira versão de `coleta.sh` executava `logcat -c`. O buffer em memória anterior a 10:30 foi perdido. O script foi corrigido para nunca limpar o buffer. Os logs de boot posteriores foram coletados por completo.
- **Captura de E1b sobrescrita.** A captura da repetição ociosa (10:44-10:59) usou o mesmo nome de arquivo da captura de boot iniciada às 11:28, que a substituiu. O log DNS do período foi preservado. Os snapshots ADB de E1b estão íntegros.
- **Primeiro reboot sem captura de rede.** O reboot de ~11:24 ocorreu depois do fim de uma captura. Restaram o log DNS e o logcat. O experimento foi repetido às 11:28 com captura.
- **Rotulagem corrigida.** A coleta das 10:45 foi rotulada inicialmente como "pós-boot". O uptime de 51 minutos mostrou que não houve reinicialização e a pasta foi renomeada para "rodada 2 ociosa".
- **Correção de interpretação.** Durante a análise de E1 os PIDs 5452 e 10321 foram interpretados como reinício do `uotaback`. A rodada E1b mostrou que eram os dois apps diferentes executando ao mesmo tempo. A análise de E2 indicou que a consulta a `uota.uota.xyz` ocorria só no boot. O log DNS completo mostrou consultas também fora do boot. As duas interpretações foram corrigidas neste relatório.
- **Correlação limitada a snapshots.** Conexões curtas que abrem e fecham entre snapshots não têm processo associado. No caso do Baidu isso foi resolvido com o experimento E5, de leitura em alta frequência. Mesmo assim, o `netstat` não distinguiu entre os dois processos UOTA, que compartilham o UID 1000.
- **E6 com leitura parcial.** O USB caiu às 16:00:19 e a leitura de sockets cobriu apenas o primeiro ciclo de NTP. Os ciclos seguintes foram atribuídos ao UOTA pela repetição da mesma sequência de destinos. Os PIDs foram associados ao par UOTA pelo nome truncado do `netstat`, sem confirmação por `ps`.
- **Atribuição do NTP do sistema.** Até o E6 todo o tráfego NTP havia sido atribuído ao `system_server` apenas pelo nome do domínio. O E6 e a lista encontrada pelo Grupo 1 mostraram que quase todo ele é do UOTA, inclusive `time.google.com`. Só `2.android.pool.ntp.org` segue atribuído ao sistema, por inferência.
- **Hash divergente em `etapa0/coletado_em.txt`.** O Grupo 1 verificou os hashes recebidos e encontrou divergência nesse arquivo. A causa foi um erro de ordem no comando de coleta: o `SHA256SUMS` foi gerado às 10:27:05.532 com uma versão anterior do arquivo, que foi reescrito às 10:27:05.538. O conteúdo atual registra a hora da coleta (10:27:05). Os demais arquivos da pasta conferem.
- **Hash do log de DNS.** Uma versão anterior deste relatório trazia o hash do `dnsmasq.log` calculado às 13:20, mas o log continuou recebendo consultas até 19:32. O Apêndice A traz

agora o hash da versão final, a mesma enviada aos outros grupos.

- **Hardening sem captura de tráfego.** O Grupo 3 não capturou tráfego depois das medidas e usou outra unidade com o mesmo firmware. O desaparecimento do tráfego do UOTA é inferido pela ausência dos processos, não medido.
- **Unidade compartilhada.** A proposta recomenda unidades separadas por grupo. A mesma unidade foi usada pelo Grupo 1. A seção 11 lista as alterações.
- **Amostra.** Uma única unidade foi analisada. Os resultados valem para esta variante de firmware (`dolphin-fvd-p1`, build 20201214173318) e não podem ser generalizados para o lote sem a Etapa 0 em outras unidades.
- **Relógio da box.** O relógio volta a 31/12, 19:00, a cada boot. Os horários do logcat foram convertidos usando o horário do notebook.

11: Alterações realizadas na unidade

Horário	Ação	Persistente	Situação
~10:10	Box conectada à rede Wi-Fi `lab-tvbox`	sim (rede salva)	a remover antes da entrega ao Grupo 1
~10:20	Acesso às Opções do desenvolvedor; "Depuração USB" e "usb0 device mode" já estavam ativos	-	sem mudança
10:26	`adb tcpip 5555`	não	desfeito pelo reboot
10:30	`logcat -c`	não (memória)	log anterior perdido
11:24, 11:28, ~12:00, ~12:29	Reinicializações pela tomada	não	-
12:35-12:41	Abertura de Kodi, YouTube, GetApps, Play Store e Configurações	sim (cache e dados de primeira execução em `/data`)	não revertido
~12:38	Permissão de armazenamento concedida à GetApps	sim	revogada às 13:16 com `pm revoke`

Nenhum aplicativo foi instalado, removido ou desativado. Nenhuma propriedade persistente (`persist.*`) foi alterada. A lista também está em `evidencias/REGISTRO_ALTERACOES_NA_UNIDADE.md`.

12: Conclusão

A unidade analisada roda um Android 7.0 de 2016 que se apresenta como Android 11.1. É uma compilação de desenvolvimento com root aberto pelo ADB e SELinux desligado. Na rede, a maior parte do tráfego vem de serviços do Google. O comportamento relevante para a segurança vem dos atualizadores `uota` e `uotaback`. Eles executam como system e enviam identificadores persistentes do aparelho em texto claro para um servidor de terceiro escondido atrás da Cloudflare, após o boot e também em outros momentos. Esse comportamento é uma vulnerabilidade demonstrada. A análise do Grupo 1 confirmou que a resposta desse servidor pode levar à instalação silenciosa de pacotes e à execução de comandos, sem autenticação do emissor. Não foi demonstrado que seja malicioso, porque nenhum conteúdo enviado pelo servidor foi observado.

Para o reaproveitamento educacional, os resultados indicam que a unidade não deve ser ligada a uma rede institucional no estado em que se encontra. No mínimo, os domínios e apps listados na seção 9.4 devem ser bloqueados ou desativados e o resultado deve ser verificado repetindo os experimentos deste relatório. O Grupo 3 mostrou que desativar o UOTA e o controle remoto elimina os processos e as portas expostas sem afetar imagem, som e vídeo. Ficam como questões abertas o conteúdo que o servidor real enviaria, a camada nativa da GetApps e a confirmação por captura de tráfego de que os destinos do UOTA desaparecem.

Apêndice A: evidências e hashes

Todos os caminhos são relativos a `evidencias/`. Cada pasta em `dinamica/` e `sinkhole/` também tem um arquivo `SHA256SUMS` com o hash de cada arquivo interno.

Arquivo	SHA-256
`captura_ociosa_sem_internet.pcap`	`cb88a2fd475155cb375593e93a7d6018d371618195d48bead0391dbc27dd4150`
`captura_boot_sem_internet.pcap`	`34b54e93326c39d3068a351c5a4f639065b5aa3b659e245e366ce2f07d6401fd`
`captura_interacao_sem_internet.pcap`	`e19dc6b63cb2fbf6ca45f273bc9b18fceca55b96e2223818545f87ea320f6f9f`
`dns/dnsmasq.log` (9/10, 10:11 a 19:32)	`3677c348eb3a9c409d0870be59980dab21ec26d03162400777d81752705a6e61`
`sinkhole/requisicoes.jsonl`	`5ef6fe520fe273d8c17c3ab27723191ed9188928b0e2368e6135baa8287da9ba`
`etapa0/getprop.txt`	`6cffe6441be88ade033bfa83920aa6a449109192150d54a332dc2fa4e060adcd`
`etapa0/pacotes.txt`	`188af7d25edfad5199cb05bc1d6158a8ba956f8a4025a98a711734f28b35cf54`
`dinamica/caca_porta80_20261009_144024/conexoes_porta80.txt`	`bd02c62025d730a8f344e6892c582f91d0c6f666001dfffb14a586d8655da68d`
`captura_ntp_sem_internet.pcap`	`30426f7cb538f869e2b7f33c75e7cd2a61f4b3611aca66f2153c65f034a8f2ec`
`dinamica/caca_ntp_20261009_155526/sockets_ntp.txt`	`aa9af6173859d385985fa9840a58376451c674b4caab9144b1434941916b0a98`
`dinamica/boot2_com_captura/logcat_desde_boot.txt`	`fdadb7ad46393dfe9591fa2d21eb3517a8cd888514c791921680f92285af23a3`

Pasta	Conteúdo
`etapa0/`	propriedades, sistema, lista de pacotes, informação do SoC
`dinamica/20261009_103058_ociosa_sem_internet/`	E1: snapshots, logcat, alarmes, jobs, RESUMO.md
`dinamica/20261009_104520_ociosa_sem_internet_rodada2/`	E1b: snapshots, NOTA.txt
`dinamica/boot1_sem_captura/`, `boot2_com_captura/`	logcat desde o boot e estado logo após o boot
`dinamica/20261009_113006_pos_boot_sem_internet/`	E2: snapshots, RESUMO.md
`sinkhole/`	E3: requisições brutas (`req_*.bin`), `requisicoes.jsonl`, RESUMO.md
`dinamica/20261009_123545_com_interacao_sem_internet/`	E4: snapshots, logcat com as telas abertas, RESUMO.md
`dinamica/caca_ntp_20261009_155526/`	E6: sockets UDP com horário e PID, lista UID → pacote, quedas do USB
`dinamica/caca_porta80_20261009_144024/`	E5: conexões para a porta 80 com horário, lista UID → pacote, `ps` final

Apêndice B: requisição capturada no sinkhole

Arquivo `sinkhole/req_20261009_123013_962398.bin`, requisição do app `com.uv.droid.uota` no boot das 12:30. As quebras de linha na URL foram inseridas para leitura.

```
GET /api/common/ota/profile/v1?did=a82009eca3c6
  &wifiMac=a4:7d:9f:24:a0:9c&uuid=&brand=Allwinner&model=TV%20BOX
  &deviceName=dolphin-fvd-p1&os=11.1&appVer=10003&appId=uotamain
  &fingerprint=Allwinner/dolphin_fvd_p1/dolphin-fvd-p1:7.0/NRD91N/20201214173318:eng/test-keys
  &cpuSerial=143078f2080c442704d3&cpuHardwareModel=sun8iw7
  &androidId=696298afbf9fba22&firmwareClient=unknownClient
  &firmwareVersion=1607938396&firmwareName=SoftFirmware
  &launcherPackageName=com.droidlogic.mboxlauncher:24;com.android.settings:24
  &clientPackageName=com.uv.droid.uota&isSystemApp=true HTTP/1.1
Host: uota.uota.xyz:2082
Connection: Keep-Alive
Accept-Encoding: gzip
User-Agent: okhttp/3.8.1
```

A requisição do `com.uv.droid.uotaback` é igual, exceto pelo caminho `/api/common/otabackup/profile/v1`, por `appVer=10002`, `appId=uotaback` e pela ausência do parâmetro `launcherPackageName`.

Apêndice C: scripts utilizados

Arquivo	Função
`lab/hotspot.sh`	`up` / `down` do ponto de acesso (hostapd + dnsmasq + nftables); `internet-on` / `internet-off`; `status`
`lab/coleta.sh <rótulo> [duração] [intervalo]`	coleta somente leitura via ADB: `packages.list`, logcat (sem limpar o buffer), snapshots de `ps`, `netstat -anp`, `/proc/net/{tcp,tcp6,udp,udp6}`, `xt_qtaguid`, `dumpsys activity services`; ao final `dumpsys alarm`, `dumpsys jobscheduler` e `SHA256SUMS`
`lab/caca_ntp.sh [segundos]`	mesma lógica do `caca_porta80.sh`, registrando todos os sockets UDP exceto mDNS e DHCP
`lab/caca_porta80.sh [segundos]`	leitura de `netstat -anp` na box cerca de 3 vezes por segundo, filtrando conexões para a porta 80; reconecta quando o USB oscila
`lab/sinkhole.sh on|off`	reinicia o dnsmasq com `--address=/uota.xyz/10.42.99.1` e libera a porta 2082 no firewall
`lab/sinkhole_server.py`	servidor TCP em 10.42.99.1:2082 que grava a requisição bruta e responde `404` sem corpo

Comando de captura usado em todos os experimentos:

```
sudo tcpdump -i ap0 -Z vyvy -w evidencias/<nome>.pcap -G 600 -W 1   # -G 900 no E2
```